Name :R.K Ayushman Singh

Roll no:261100690014

PROCESS CONTROL LAB

Exercise 1: Viewing and ranking processes

Display all running processes with their PID, PPID, owner, CPU usage, memory usage, and command.

```
anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ps -eo pid,ppid,user,%cpu,%mem,cmd | head -n 8
  PID  PPID USER     %CPU %MEM CMD
    1     0 root      0.7  0.1 /process_api --firecracker-init --addr 0.0.0.0:2024 --max-ws-buffer-size 32768 --block-local-connections --control-vsock-port 5001 --listen-vsock-port 2024 --log-vsock-port 5002
    2     0 root      0.0  0.0 [kthreadd]
    3     2 root      0.0  0.0 [pool_workqueue_release]
    4     2 root      0.0  0.0 [kworker/R-rcu_gp]
    5     2 root      0.0  0.0 [kworker/R-sync_wq]
    6     2 root      0.0  0.0 [kworker/R-kvfree_rcu_reclaim]
    7     2 root      0.0  0.0 [kworker/R-slub_flushwq]

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ _
```

List only the processes belonging to the current user.

```
anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ps -u $USER -o pid,ppid,user,%cpu,%mem,cmd
  PID  PPID USER     %CPU %MEM CMD
  110   109 anonymo+  0.0  0.0 bash --noprofile --norc /tmp/steps.sh
  123   110 anonymo+  0.0  0.1 ps -u anonymous -o pid,ppid,user,%cpu,%mem,cmd

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ _
```

Identify the five processes consuming the most CPU.

```
anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ps -eo pid,ppid,user,%cpu,%mem,cmd --sort=-%cpu | head -n 6
  PID  PPID USER     %CPU %MEM CMD
  100    97 root     39.3  1.3 node proc.js
    1     0 root      0.7  0.1 /process_api --firecracker-init --addr 0.0.0.0:2024 --max-ws-buffer-size 32768 --block-local-connections --control-vsock-port 5001 --listen-vsock-port 2024 --log-vsock-port 5002
   28     2 root      0.1  0.0 [kworker/u4:1-iou_exit]
   59     1 root      0.1  0.8 /opt/rclone/rclone-filestore multimount --config /dev/shm/rclone-boot/config.json
   11     2 root      0.0  0.0 [kworker/0:1-events]

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ _
```

Identify the five processes consuming the most memory.

```
anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ps -eo pid,ppid,user,%cpu,%mem,cmd --sort=-%mem | head -n 6
  PID  PPID USER     %CPU %MEM CMD
  100    97 root     39.3  1.3 node proc.js
   59     1 root      0.1  0.8 /opt/rclone/rclone-filestore multimount --config /dev/shm/rclone-boot/config.json
    1     0 root      0.7  0.1 /process_api --firecracker-init --addr 0.0.0.0:2024 --max-ws-buffer-size 32768 --block-local-connections --control-vsock-port 5001 --listen-vsock-port 2024 --log-vsock-port 5002
  126   110 anonymo+  0.0  0.1 ps -eo pid,ppid,user,%cpu,%mem,cmd --sort=-%mem
  110   109 anonymo+  0.0  0.0 bash --noprofile --norc /tmp/steps.sh

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ _
```

Explain why high CPU usage alone does not confirm malicious activity.

A busy CPU is normal for many harmless jobs, such as compiling code, converting video, running a virus scan, building a search index or installing updates. A high percentage therefore only tells us that a process is working hard, not that it is hostile.

To judge whether a process is suspicious, I would also look at who owns it, which parent started it, where its binary is stored, when it started, whether it has network connections, and whether its name or path looks unusual.

Exercise 2: Parent and child processes

Write a shell script that starts two background processes and displays their PIDs.

nano family.sh

```
#!/bin/bash
echo "Parent script PID: $$"
echo "Parent script PPID: $PPID"

sleep 240 &
echo "Worker A PID: $!"

sleep 360 &
echo "Worker B PID: $!"

echo "Processes started by this script:"
ps -o pid,ppid,cmd --ppid $$

wait
```

chmod +x family.sh

```
anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ./family.sh &
[1] 128
Parent script PID: 128
Parent script PPID: 110
Worker A PID: 130
Worker B PID: 131
Processes started by this script:
  PID  PPID CMD
  130   128 sleep 240
  131   128 sleep 360
  132   128 ps -o pid,ppid,cmd --ppid 128

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ _
```

Identify the PID and PPID of the shell script.

Display the child processes created by the script.

```
anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ps -p $FPID -o pid,ppid,cmd
  PID  PPID CMD
  128   110 /bin/bash ./family.sh

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ps -o pid,ppid,cmd --ppid $FPID
  PID  PPID CMD
  130   128 sleep 240
  131   128 sleep 360

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ _
```

Explain the purpose of $$, $!, &, and wait.

$$ holds the PID of the shell or script that is currently running.

$! holds the PID of the background process that was started most recently.

& at the end of a command sends it to the background so the shell can carry on immediately.

wait pauses the script until its background child processes have finished.

Observe what happens to the child processes when the parent script terminates.

```
anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ kill $FPID

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ps -eo pid,ppid,cmd | grep "[s]leep"
```

When the parent script was killed, its two sleep processes were not killed with it. They kept running and were adopted by another process, so their PPID changed from the script's PID to a new parent.

Exercise 3: Process control using kill

Start a background process and record its PID.

Suspend the process and verify its state.

Resume the process and verify its state.

Terminate the process gracefully.

Verify that the process has terminated.

```
anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ sleep 600 &
[1] 140

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ PID=$!

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ echo $PID
140

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ kill -SIGSTOP $PID

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ps -o pid,stat,cmd -p $PID
  PID STAT CMD
  140 T    sleep 600

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ kill -SIGCONT $PID

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ps -o pid,stat,cmd -p $PID
  PID STAT CMD
  140 S    sleep 600

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ kill -SIGTERM $PID

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ps -p $PID
  PID TTY          TIME CMD

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ _
```

Explain the differences between SIGSTOP, SIGCONT, SIGTERM, and SIGKILL.

Signal	**Explanation**
SIGSTOP	Pauses (suspends) a process. It cannot be caught, blocked or ignored.
SIGCONT	Continues a process that was previously stopped.

SIGTERM	Politely asks a process to exit, so it can run its cleanup code first.
SIGKILL	Forces the kernel to end the process immediately. It cannot be caught, blocked or ignored.

Exercise 4: Graceful and forced termination

Write a monitoring script that periodically records a timestamp in a log file.

Configure the script to record a shutdown message when it receives SIGTERM.

nano watcher.sh

```
#!/bin/bash
LOGFILE="$HOME/proc_lab/watcher.log"

trap 'echo "$(date +"%F %T") SIGTERM caught, stopping cleanly" >> "$LOGFILE"; exit 0' TERM

echo "$(date +"%F %T") watcher started, PID $$" >> "$LOGFILE"

while true
do
   echo "$(date +"%F %T") alive" >> "$LOGFILE"
   sleep 2 &
   wait $!
done
```

chmod +x watcher.sh

Terminate the script using SIGTERM and inspect the log.

```
anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ./watcher.sh &
[1] 144

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ MPID=$!

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ sleep 7

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ kill -TERM $MPID

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ cat ~/proc_lab/watcher.log
2026-11-05 13:02:17 watcher started, PID 144
2026-11-05 13:02:17 alive
2026-11-05 13:02:19 alive
2026-11-05 13:02:21 alive
2026-11-05 13:02:23 alive
2026-11-05 13:02:24 SIGTERM caught, stopping cleanly

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ _
```

Restart the script and terminate it using SIGKILL.

```
anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ./watcher.sh &
[1] 158

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ MPID=$!

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ sleep 7

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ kill -KILL $MPID

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ tail -n 8 ~/proc_lab/watcher.log
2026-11-05 13:02:21 alive
2026-11-05 13:02:23 alive
2026-11-05 13:02:24 SIGTERM caught, stopping cleanly
2026-11-05 13:02:24 watcher started, PID 158
2026-11-05 13:02:24 alive
2026-11-05 13:02:26 alive
2026-11-05 13:02:28 alive
2026-11-05 13:02:30 alive

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ _
```

Compare the results and explain why they differ.

After SIGTERM the log ends with the shutdown line, because the script is allowed to catch that signal and the trap handler writes the message before exiting.

After SIGKILL there is no shutdown line. The kernel removes the process straight away, so the trap handler never gets a chance to run and the log simply stops.

Exercise 5: Multiple processes using killall

Start three instances of a lab program with a unique process name.

```
cp "$(command -v sleep)" labtask
chmod +x labtask
```

```
anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ./labtask 900 &
[1] 171

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ./labtask 900 &
[2] 172

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ./labtask 900 &
[3] 173

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ _
```

Display the PID and owner of each instance.

```
anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ps -C labtask -o pid,user,cmd
  PID USER     CMD
  171 anonymo+ ./labtask 900
  172 anonymo+ ./labtask 900
  173 anonymo+ ./labtask 900

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ _
```

Terminate one instance using its PID.

```
anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ kill $P1

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ps -C labtask -o pid,user,cmd
  PID USER     CMD
  172 anonymo+ ./labtask 900
  173 anonymo+ ./labtask 900

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ _
```

Terminate the remaining instances using killall.

Verify that all instances have terminated.

```
anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ killall labtask

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ pgrep -a labtask

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ _
```

Explain the risk of using killall with a common process name.

killall ends every process whose name matches, not just the one we have in mind. If the name is a common one such as python, bash, java or sshd, it can stop unrelated programs or even important services. A unique name should be used, and the matching processes should be listed first (for example with pgrep -a) before anything is terminated.

Exercise 6: Recurring process monitoring using cron

Write a shell script that records the current date, time, username, and number of processes owned by the current user.

```
nano cron_stats.sh
```

```
#!/bin/bash
echo "$(date '+%F %T') user=$(whoami) procs=$(ps -u "$(whoami)" --no-headers | wc -l)"
>> "$HOME/proc_lab/cron_stats.log"
```

```
chmod +x cron_stats.sh
```

Test run of the script:

```
anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ./cron_stats.sh

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ cat ~/proc_lab/cron_stats.log
2026-11-05 13:02:32 user=anonymous procs=6

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ _
```

Schedule the script to run every minute.

```
crontab -e
```

Line added to the crontab:

```
* * * * * /home/anonymous/proc_lab/cron_stats.sh
```

```
crontab -l
```

Verify that the scheduled task executes successfully.

```
systemctl status cron
tail -f ~/proc_lab/cron_stats.log
```

Start additional background processes and observe changes in the log.

```
sleep 500 &
sleep 500 &
```

The processes value in the next log lines should increase by the number of extra sleep processes started.

Modify the schedule to run every five minutes.

```
crontab -e
```

```
*/5 * * * * /home/anonymous/proc_lab/cron_stats.sh
```

Remove the exercise entry without deleting other cron entries.

```
crontab -l | grep -v "cron_stats.sh" | crontab -
crontab -l
```

Exercise 7: Cron scheduling

Write cron entries for the following requirements:

Requirement	Cron entry
Check failed-login logs every five minutes	*/5 * * * * /usr/local/bin/check_failed_logins.sh
Capture a process snapshot at the beginning of every hour	0 * * * * /usr/local/bin/process_snapshot.sh
Back up security logs daily at 2:30 AM	30 2 * * * /usr/local/bin/backup_security_logs.sh
Generate a security report every weekday at 9:00 AM	0 9 * * 1-5 /usr/local/bin/security_report.sh
Perform a file-integrity check every Sunday at 3:00 AM	0 3 * * 0 /usr/local/bin/integrity_check.sh
Run a monitoring script every 15 minutes	*/15 * * * * /usr/local/bin/run_monitor.sh
Generate a weekly report every Monday at 8:30 AM	30 8 * * 1 /usr/local/bin/weekly_report.sh

Field order in each entry: minute, hour, day of month, month, day of week, followed by the command.

Exercise 8: One-time scheduling using at

Schedule a task to record the current date and process list two minutes from now.

```
echo "date >> $HOME/proc_lab/at_output.txt; ps aux >> $HOME/proc_lab/at_output.txt" | at now + 2 minutes
```

Display all pending jobs.

```
atq
```

Inspect the commands associated with the scheduled job.

```
at -c <JOB_NUMBER>
```

Verify that the output file is created after execution.

```
ls -l ~/proc_lab/at_output.txt
head -n 5 ~/proc_lab/at_output.txt
```

Explain why the completed job no longer appears in the pending-job list.

at is meant for one-time jobs. Once the job has run, it is deleted from the queue, so atq only lists jobs that are still waiting to be executed.

Exercise 9: Cancelling scheduled jobs

Schedule a harmless task to execute ten minutes from now.

```
echo "echo hello >> $HOME/proc_lab/harmless.txt" | at now + 10 minutes
```

Identify its job number.

```
atq
```

Cancel the scheduled task.

```
atrm <JOB_NUMBER>
```

Verify that the task has been removed.

```
atq
```

Explain the difference between cancelling an at job and removing a cron entry.

An at job runs only once and is identified by a job number, so it is cancelled with atrm. A cron entry repeats on a schedule and lives in the user's crontab, so it has to be deleted by editing the crontab; until then it keeps running at every scheduled time.

Exercise 10: Incident-response scenario

Identify all instances of the lab process and record their PIDs and owners.

```
anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ./labtask 900 &
[1] 190

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ./labtask 900 &
[2] 191

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ./labtask 900 &
[3] 192

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ps -eo pid,ppid,user,stat,cmd | grep "[l]abtask"
```

Suspend one instance and verify its state.

Resume the suspended instance.

Terminate all instances of the lab process.

```
anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ kill -SIGSTOP $T1

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ps -o pid,stat,cmd -p $T1
  PID STAT CMD
  190 T    ./labtask 900

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ kill -SIGCONT $T1

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ ps -o pid,stat,cmd -p $T1
  PID STAT CMD
  190 S    ./labtask 900

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ pkill -TERM -x labtask

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ pgrep -a -x labtask

anonymous@anonymous-supreme in ~/proc_lab via v3.14.7
❯ _
```

Identify and inspect the scheduled restart task.

```
atq
at -c <JOB_NUMBER>
crontab -l
```

Cancel the scheduled restart.

```
atrm <JOB_NUMBER>
```

Verify that no instances or restart jobs remain.

```
pgrep -a -x labtask
atq
crontab -l
```